CSC 548- Parallel Systems

Sanjaey Shunmuga Sundaram - sshunmu

Midterm Project Report

SUMMA-based Dense Matrix-Matrix Multiplication (GEMM)

Introduction:

Matrix multiplication is an important and fundamental operation in many fields, including scientific computing, machine learning, and high-performance computing (HPC). However for large matrices, traditional methods of matrix multiplication such as serial computation becomes laborious and time consuming, due to its $O(n^3)$ time complexity. To mitigate this, we leverage parallel computing using MPI (Message passing Interface) to distribute a matrix in blocks across multiple processors, compute them and aggregate the results.

This project implements the Scalable Universal Matrix Multiplication Algorithm (SUMMA), which is an efficient block-based, communication - optimized parallel matrix multiplication algorithm. The SUMMA algorithm uses broadcast based communication, reducing the dependencies between the processes, and allows efficient computation and execution on clusters. In this project I have implemented two different variants of the SUMMA algorithm:

- SUMMA with stationary A (summa_stationary_a)
    - Basically a matrix multiplication involves 2 matrices, namely Matrix A and Matrix B which when multiplied, yields the resultant matrix i.e, Matrix C.
    - In this algorithm, Matrix A is kept stationary, and Matrix B is dynamically broadcasted column wise across processes.
    - This algorithm is optimized for scenarios where A is large and B is relatively smaller.

- SUMMA with Stationary C (summa_stationary_c)
    - In this algorithm, contrary to stationary A, both Matrix A and Matrix B are broadcasted among processes.
    - Matrix A is broken down into blocks, and broadcasted along rows, and Matrix B is broken down into blocks, and broadcasted along columns, and accumulates parietal results into Matrix C.
    - This algorithm is suitable for balanced workloads.

In this implementation, each variant uses separate data distribution strategies, to optimize communication, minimize memory overheads, and to ensure accurate and efficient execution.

Implementation:

Grid Setup:

A 2-Dimensional Cartesian process grid was used instead of a 1-Dimensional linear arrangement. A 2D grid was chosen because it minimizes the communication overhead, as this allows to simultaneously broadcast the blocks of the matrices in row-wise and column-wise. It also ensures balanced load across processors and also provides a structure for cache-coherency.

```C/C++
int grid_size = (int)sqrt(nprocs);
int dims[2] = {grid_size, grid_size};
int periods[2] = {0, 0};
MPI_Comm grid_comm;
MPI_Cart_create(MPI_COMM_WORLD, 2, dims, periods, 1, &grid_comm);
```

Data Distribution Strategy, Communication and Implementation:

Since SUMMA A and SUMMA C differ in how they distribute data and perform communication, we have designed two separate matrix distribution functions:

The distribute_matrix_blocks function efficiently distributes submatrices of A and B to MPI processes in a 2 Dimensional grid. It first computes the block sizes to evenly distribute the matrices among processes.

On rank 0, it packs matrix blocks into continuous buffers i.e A_packed and B_packed using memcpy(), ensuring each process gets the correct submatrix. These packed blocks are then scattered to all processes using MPI_Scatter, with each process receiving its portion into A_local and B_local. After distribution, rank 0 frees the packed buffers to prevent memory leaks and MPI_Barrier ensures synchronization before proceeding.

C/C++

```
void distribute_matrix_blocks(float *A, float *B, float *A_local, float *B_local,
int m, int n, int k, int rank, int grid_size, MPI_Comm comm) {


 // TODO: Implement matrix block distribution logic
 //Define the function signature and everything else
 int block_m = (int)ceil((double)m / grid_size);
 int block_n = (int)ceil((double)n / grid_size);
 int block_k = (int)ceil((double)k / grid_size);

 float *A_packed = NULL;
 float *B_packed = NULL;

 if (rank == 0) {
   A_packed = (float *)malloc(m * k * sizeof(float));
   B_packed = (float *)malloc(k * n * sizeof(float));
   // Manually copying blocks into contiguous arrays
   for (int i = 0; i < grid_size; i++) {
     for (int j = 0; j < grid_size; j++) {
         int dest_idx = (i * grid_size + j) * block_m * block_k;
         int src_row = i * block_m;
         int src_col = j * block_k;

         for (int r = 0; r < block_m; r++) {
             memcpy(&A_packed[dest_idx + r * block_k], &A[(src_row + r) * k +
src_col], block_k * sizeof(float));
         }
     }
 }

 for (int i = 0; i < grid_size; i++) {
     for (int j = 0; j < grid_size; j++) {
         int dest_idx = (i * grid_size + j) * block_k * block_n;
         int src_row = i * block_k;
         int src_col = j * block_n;

         for (int r = 0; r < block_k; r++) {
             memcpy(&B_packed[dest_idx + r * block_n], &B[(src_row + r) * n +
src_col], block_n * sizeof(float));
         }
     }
 }


}
```

```
 MPI_Scatter(A_packed, block_m * block_k, MPI_FLOAT, A_local, block_m * block_k,
MPI_FLOAT, 0, comm);
 MPI_Scatter(B_packed, block_m * block_k, MPI_FLOAT, B_local, block_m * block_k,
MPI_FLOAT, 0, comm);

 if (rank == 0) {
   free(A_packed);
   free(B_packed);
}
  MPI_Barrier(comm);
   printf("\n");

   MPI_Barrier(comm);

}
```

The distribute_matrix_blocks_a function distributes submatrices of A and B from the root process (rank 0) to all MPI processes for SUMMA stationary A matrix multiplication. The function ensures that matrix B is distributed row-wise, where rank 0 sends chunks of B to the respective process using MPI_Send and the other portions with MPI_Recv. Similarly, matrix A is divided into blocks and rank 0 sends block-wise data to the appropriate processes. After receiving the A blocks, processes within the same row exchange their A_local_block using MPI_Allgather, which ensures that all processes in a row have access to the required parts of A. A final MPI_Barrier synchronizes the processes before computation begins.

```C/C++
void distribute_matrix_blocks_a(int rank, float* A, float* B, float*
A_local_block, float* A_local_slab, float* B_local, int p, int m, int n, int k,
int block_m, int block_n, int block_k, float* B_temp, MPI_Comm row_comm)
{
 if (rank == 0)
 {
   //printf("Matrix dimension k: %d, Block size block_k: %d\n", k, block_k);

   for (int row = 0; row < k; row++)
```

```
   {
     for (int i = 0; i < p; i++)
     {
       int y = row / block_k;
       int x = i;
       if ( y*p + x == 0)
       {
         memcpy(B_local + row * block_n, B + row * n + i * block_n, block_n *
sizeof(float));
       }
       MPI_Send( B + row * n + i * block_n, block_n, MPI_FLOAT, y*p + x, 0,
MPI_COMM_WORLD );
     }
   }
 }
 else
 {
   for (int row = 0; row < block_k; row++)
   {
     MPI_Recv( B_local + row * block_n, block_n, MPI_FLOAT, 0, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
   }
 }
  MPI_Barrier(MPI_COMM_WORLD);
  if (rank == 0)
 {
   for (int row = 0; row < m; row++)
   {
     for (int i = 0; i < p; i++)
     {
       int y = row / block_m;
       int x = i;
       if ( y*p + x == 0)
       {
         memcpy(A_local_block + row * block_k, A + row * k + i * block_k, block_k
* sizeof(float));
       }
       MPI_Send( A + row * k + i * block_k, block_k, MPI_FLOAT, y*p + x, 0,
MPI_COMM_WORLD );
     }
   }
 }
 else
 {
   for (int row = 0; row < block_m; row++)
```

```
   {
     MPI_Recv( A_local_block + row * block_k, block_k, MPI_FLOAT, 0, 0,
MPI_COMM_WORLD, MPI_STATUS_IGNORE);
   }
 }
  MPI_Barrier(MPI_COMM_WORLD);
  int chunk_size = block_m * block_k;
 MPI_Allgather(A_local_block, chunk_size, MPI_FLOAT, A_local_slab, chunk_size,
MPI_FLOAT, row_comm);
}
```

SUMMA Computation:

The summa_stationary_c function implements parallel matrix multiplication using the SUMMA with a stationary C matrix. It first initializes a 2D MPI process grid, using MPI_Cart_Create, allowing the processes to communicate efficiently. Each process calculates its local block sizes to evenly distribute matrix computation. The root process, rank 0, generates matrices A and B, which then are distributed among all processes using distribute_matrix_blocks. The SUMMA algorithm, where at each step, submatrices of A and B are broadcasted across their respective row and column communicators using MPI_Bcast. Each process performs local matrix multiplication on received data, accumulating results in C_local. Finally rank 0 gathers all computed blocks using gather_matrix_blocks to construct the final output matrix.

```C/C++
void summa_stationary_c(int m, int n, int k, int nprocs, int rank) {

 // Grid setup
 // TODO: Initialize grid variables

 int grid_size = (int)sqrt(nprocs);
 int dims[2]= {grid_size, grid_size};
 int periods[2] = {0,0};

 MPI_Comm grid_comm, row_comm, col_comm;
  // Create 2D process grid
```

```
// TODO: Create a 2D Cartesian communicator

MPI_Cart_create(MPI_COMM_WORLD, 2, dims, periods, 1, &grid_comm);


// Get process coordinates
// TODO: Get the coordinates of the process in the grid

int coords[2];
MPI_Cart_coords(grid_comm, rank, 2, coords);

// Create row and column communicators
// TODO: Create row and column communicators for the grid

int row_rank = coords[0], col_rank = coords[1];
MPI_Comm_split(grid_comm, row_rank, rank, &row_comm);
MPI_Comm_split(grid_comm, col_rank, rank, &col_comm);


// Determine local block sizes
// TODO: Calculate the local block sizes for each process

int block_m = (m + grid_size -1) / grid_size;
int block_k = (k + grid_size -1) / grid_size;
int block_n = (n + grid_size -1) / grid_size;

// Generate random matrices on root process
// TODO: Generate random matrices A and B on the root process

float *A = NULL, *B = NULL;
  if (rank == 0) {
      A = generate_matrix_A(m, k, rank);
      B = generate_matrix_B(k, n, rank);
  }




// Allocate local matrices
// TODO: Allocate memory for local matrices

float *A_local = (float *)malloc(block_m * block_k * sizeof(float));
float *B_local = (float *)malloc(block_k * block_n * sizeof(float));
float *C_local = (float *)calloc(block_m * block_n, sizeof(float));
```

```
 // Distribute matrix blocks
 // TODO: Distribute matrix blocks to all processes

 distribute_matrix_blocks(A, B, A_local, B_local, m, n, k, rank, grid_size,
grid_comm);

 // Distribute matrix blocks with proper arguments for all processes
 // TODO: Call distribute_matrix_blocks with appropriate arguments

 // SUMMA computation
 // TODO: Implement the SUMMA computation
 //You can use the function in utils.c to perform the matrix multiplication

 for (int step = 0; step < grid_size; step++) {

   float *A_temp = (float *)malloc(block_m * block_k * sizeof(float));
   float *B_temp = (float *)malloc(block_k * block_n * sizeof(float));


   if (col_rank == step) {
     memcpy(A_temp, A_local, block_m * block_k * sizeof(float));
   }
   MPI_Bcast(A_temp, block_m * block_k, MPI_FLOAT, step, row_comm);
    if (row_rank == step) {
     memcpy(B_temp, B_local, block_k * block_n * sizeof(float));
   }
   MPI_Bcast(B_temp, block_k * block_n, MPI_FLOAT, step, col_comm);


   /

   matmul(A_temp, B_temp, C_local, block_m, block_n, block_k);

   free(A_temp);
   free(B_temp);

}




 // Gather results
 // TODO: Gather the results from all processes
```

```
 float *C_global = NULL;
 if (rank == 0) {
   C_global = (float *)calloc(m * n, sizeof(float));  // Use calloc to initialize
to 0

 }



 int *recvcounts = NULL;
 int *displs = NULL;

// Rank 0 initializes the arrays
if (rank == 0) {
   recvcounts = (int *)malloc(nprocs * sizeof(int));
   displs = (int *)malloc(nprocs * sizeof(int));

   for (int i = 0; i < nprocs; i++) {
       recvcounts[i] = block_m * block_n;
       displs[i] = (i / grid_size) * block_m * n + (i % grid_size) * block_n;
   }
 }


 MPI_Barrier(MPI_COMM_WORLD); // Ensure all processes have finished computing
 gather_matrix_blocks(C_local, C_global, m, n, block_m, block_n, nprocs,
grid_comm);

 // Verify results
 // TODO: Verify the correctness of the results
 //You can use the function in utils.c to verify the results



 free(A_local);
 free(B_local);
 free(C_local);

 if (rank == 0) {
   free(A);
   free(B);
   free(C_global);
   free(recvcounts);
   free(displs);
```

```
   }

}
```

The summa_stationary_a function implements parallel matrix multiplication partially similar to that of the stationary c function. After initialization of 2D process grid, calculation of local block sizes and matrix generation, distribute_matrix_blocks_a function is used to distribute matrix blocks. During the SUMMA computation, matrix B is broadcasted column-wise while matrix A remains stationary, allowing each process to compute its portion of matrix C in parallel. A final gather operation collects the computed submatrices into the final matrix C on the root process.

```C/C++
void summa_stationary_a(int m, int n, int k, int total_process_count, int rank)
{
 MPI_Comm grid_comm, col_comm, row_comm;
 int num_dimensions = 2;
 int p = sqrt(total_process_count);
 int dims[2] = {p,p};
 int periods[2] = {0, 0};
 MPI_Cart_create(MPI_COMM_WORLD, num_dimensions, dims, periods, 0, &grid_comm);
 int coords[2];
 MPI_Cart_coords(grid_comm, rank, num_dimensions, coords);
 int color = rank % p;
 MPI_Comm_split(grid_comm, color, rank, &col_comm);
 color = rank / p;
 MPI_Comm_split(grid_comm, color, rank, &row_comm);

 int block_m = ceil(m/p);
 int block_n = ceil(n/p);
 int block_k = ceil(k/p);
 float *A, *B;
 if (rank == 0)
 {
   A = generate_matrix_A(m, k, rank);
   B = generate_matrix_B(k, n, rank);
 }
```

```
 int B_mem_size = block_k * block_n;
 int A_block_mem_size = block_m * block_k;
 int A_rect_mem_size = block_m * k;
 int C_mem_size = block_m * block_n;
 float* B_local = (float*) calloc(B_mem_size, sizeof(float));
 float* B_temp = (float*) calloc(B_mem_size, sizeof(float));
 float* A_local_block = (float*) calloc(A_block_mem_size, sizeof(float));
 float* A_local_rect = (float*) calloc(A_rect_mem_size, sizeof(float));
 float* C_local = (float*) calloc(C_mem_size, sizeof(float));
 float* C = (float*) calloc(m*n, sizeof(float));
 distribute_matrix_blocks_a(rank, A, B, A_local_block, A_local_rect, B_local, p,
m, n, k, block_m, block_n, block_k, B_temp, row_comm);
 MPI_Barrier(MPI_COMM_WORLD);

 for (int j=0; j<p; j++)
 {
   if (coords[0] == j)
   {
     memcpy( B_temp, B_local, block_k * block_n * sizeof(float) );
   }
   MPI_Bcast( B_temp, block_k * block_n, MPI_FLOAT, j, col_comm );
   matmul(A_local_rect + j * block_m * block_k, B_temp, C_local, block_m, block_n,
block_k);
   MPI_Barrier(MPI_COMM_WORLD);
 }

 MPI_Barrier(MPI_COMM_WORLD);
 gather_matrix_blocks(C_local, C, m, n, block_m, block_n, total_process_count,
grid_comm);
if (rank == 0)
 {
   verify_result(C, A, B, m, n, k);
 }
 MPI_Barrier(MPI_COMM_WORLD);
}
```

The gather_matrix_blocks function collects distributed matrix blocks into a global matrix on rank 0 using MPI_Gatherv. It first defines a custom MPI subarray datatype (block_type) to ensure correct block placement. The root process allocates recvcounts and displacements, mapping each process's block to its position in the full matrix. MPI_Gatherv then gathers all submatrices efficiently. Finally, allocated memory and the MPI datatype are freed.

C/C++

```
void gather_matrix_blocks(float *local_matrix, float *matrix, int full_rows, int
full_cols, int block_rows, int block_cols, int num_process, MPI_Comm grid_comm) {
 MPI_Datatype block_type;
 {
     MPI_Datatype temp_type;
     int sizes[2] = {full_rows, full_cols};
     int subsizes[2] = {block_rows, block_cols};
     int starts[2] = {0, 0};
     MPI_Type_create_subarray(2, sizes, subsizes, starts, MPI_ORDER_C, MPI_FLOAT,
&temp_type);
     MPI_Type_create_resized(temp_type, 0, sizeof(float), &block_type);
     MPI_Type_commit(&block_type);
     MPI_Type_free(&temp_type);
 }
 int *recvcounts = NULL;
 int *displacements = NULL;
 if (matrix != NULL) {
     recvcounts = (int *)malloc(num_process * sizeof(int));
     displacements = (int *)malloc(num_process * sizeof(int));
     if (!recvcounts || !displacements) {
         printf("Error: Memory allocation failed for gather parameters.\n");
         MPI_Abort(grid_comm, 1);
     }
     for (int i = 0; i < num_process; i++) {
         int proc_coords[2];
         MPI_Cart_coords(grid_comm, i, 2, proc_coords);
         recvcounts[i] = 1;
         int row_idx = proc_coords[0] * block_rows;
         int col_idx = proc_coords[1] * block_cols;
         displacements[i] = row_idx * full_cols + col_idx;
     }
 }
 MPI_Gatherv(local_matrix, block_rows * block_cols, MPI_FLOAT, matrix, recvcounts,
displacements, block_type, 0, grid_comm);
 if (matrix != NULL) {
     free(recvcounts);
     free(displacements);
 }
 MPI_Type_free(&block_type);
}
```

Challenges Faced and Solutions:

- **Efficient Data Distribution:**
  - One of the main challenges was Non- Contiguous memory layout and Incorrect data assignment. Initially the MPI_Scatter function did not work properly when distributing matrix blocks because the data was not stored contiguously in the memory.
  - A and B matrices were stored in row - major order, while submatrices extracted for each process were not contiguous. This caused an issue where processes received incorrect matrix blocks or data was misaligned.
  - Solution was instead of directly scattering the data from global matrices, we have manually packed matrix blocks into contiguous arrays before calling the MPI_Scatter function.
  - We Extracted submatrices from A and B and stored them in contiguous memory buffers (A_packed and B_packed). Used memcpy() to ensure that each block was correctly aligned before scattering
  - After packing, the contiguous data buffers were correctly distributed using MPI_Scatter.
  - This Ensured that each process received correctly aligned submatrices. Improved the performance of matrix access by avoiding random memory jumps. It also Reduced communication overhead by ensuring that each process receives only its relevant data.

```
C/C++
if (rank == 0) {
   A_packed = (float *)malloc(m * k * sizeof(float));
   B_packed = (float *)malloc(k * n * sizeof(float));
   // Manually copying blocks into contiguous arrays
   for (int i = 0; i < grid_size; i++) {
     for (int j = 0; j < grid_size; j++) {
         int dest_idx = (i * grid_size + j) * block_m * block_k;
         int src_row = i * block_m;
         int src_col = j * block_k;

         for (int r = 0; r < block_m; r++) {
             memcpy(&A_packed[dest_idx + r * block_k], &A[(src_row + r) * k +
src_col], block_k * sizeof(float));
         }
     }
 }

 for (int i = 0; i < grid_size; i++) {
```

```
    for (int j = 0; j < grid_size; j++) {
        int dest_idx = (i * grid_size + j) * block_k * block_n;
        int src_row = i * block_k;
        int src_col = j * block_n;

        for (int r = 0; r < block_k; r++) {
            memcpy(&B_packed[dest_idx + r * block_n], &B[(src_row + r) * n +
src_col], block_n * sizeof(float));
        }
    }
 }

}

MPI_Scatter(A_packed, block_m * block_k, MPI_FLOAT, A_local, block_m * block_k,
MPI_FLOAT, 0, comm);
MPI_Scatter(B_packed, block_m * block_k, MPI_FLOAT, B_local, block_m * block_k,
MPI_FLOAT, 0, comm);
```

- **Balancing Computation and Communication:**
    - Since matrices were divided into block partitions, some processes were assigned more work than others, especially when m, n, or k were not perfectly divisible by the grid size. This led to some processes finishing earlier while others were still computing, causing an imbalance in execution time.
    - Used ceil(m / grid_size) to ensure even distribution of matrix blocks among processes. If some processes had extra work, we padded A and B matrices with zeros to maintain uniform block sizes.
    - This prevented idle waiting of certain processes, ensured all processes completed simultaneously and improved parallel efficiency. This solution avoided uneven computation loads across the grid.

- **Reducing communication overhead:**
    - Another challenge was excessive data movement and redundant communication.
    - Initially, every process was performing redundant memory operations, leading to high communication overhead. Stationary A used MPI_Send multiple times, causing latency issues. Stationary C relied on MPI_Bcast, which was optimized but still costly when k was large.
    - As a solution, we Replaced MPI_Send with MPI_Allgather for Stationary A, ensuring less overhead in distributing data and optimized MPI_Bcast calls for

Stationary C to ensure that each column and row communicator only broadcasted once per step.
    - This Reduced message count from $O(p^2)$ to $O(p)$, significantly reducing communication time. Ensured that each process only received the necessary data instead of unnecessary broadcasts.

- **Memory Constraints:**
    - When testing large matrices (e.g., 32,768 x 32,768), processes ran out of memory due to excessive allocation of temporary buffers (A_temp, B_temp, C_local). Multiple copies of matrices increased memory usage.
    - To mitigate this I used MPI_Bcast with in-place buffers instead of creating new A_temp and B_temp arrays. Reduced memory footprint by freeing buffers immediately after use.
    - This allowed for successful execution of large matrices without memory failures and reduced unnecessary allocations, improving memory efficiency.

- **Mismatched Matrix block in the Final output:**
    - Using MPI_Gather directly led to incorrect ordering of matrix blocks in the final result C_global. Processes stored blocks in a non-contiguous way, which MPI_Gather could not correctly reconstruct.
    - Replaced MPI_Gather with MPI_Gatherv, which allows specifying displacement offsets. Used MPI_Cart_coords to compute exact block placements in the global matrix.
    - Ensured correct block placement in C_global and eliminated misaligned submatrices, making verification more accurate.

Testing and Test Environment Details:

Hardware Configuration:

The development, and testing were conducted on the ARC cluster and also the Hazel Linux clusters. The following are its specifications:

- **ARC Cluster**:
    - Total Computing Resources: 80 nodes, 1280 cores
    - Processor Architectures :
        - Intel Broadwell: nodes c[50-59], queue: -p broadwell
        - Intel Skylake Silver: nodes c[0-19,26-29], queue: -p skylake
        - AMD Epyc Rome: nodes c[20-25,33,37-48,73-79], queue: -p rome

        - Intel Cascade Lake: nodes c[35-36], queue: -p cascade
        - AMD Epyc Milan: nodes c[30-31,34], queue: -p milan
        - AMD Epyc Siena: nodes c[60-72], queue: -p siena
        - AMD Epyc Genoa: nodes c[32,49], queue: -p genoa
        - AMD Epyc Turin: node c[28], queue: -p turin
    - Memory Configuration: 64GB - 128GB DDR4 ECC DRAM per node.
    - Job Scheduling: Slurm workload manager.

- **Hazel Cluster**:
    - Total Computing Resources: 400 nodes, 14000+ cores
    - Processor Architectures : Intel Xeon-based dual-socket servers.
    - Memory Configuration: 128GB - 1024GB RAM
    - Storage: 1GB directory to store scripts and Executables
    - Job Scheduling: LSF queueing system

Software Configuration:

- Operating System: Linux - Based.
- Programming models:
    - OpenMP : Shared memory parallel programming
    - MPI: OpenMPI for distributed parallelism.
- Compilers: mpicc for mpi programs.
- Job Execution:
    - Resource Request: salloc -n 16 # get 16 cores (1 node) in interactive mode
    - Batch execution: sbatch job.mpi
    - Monitoring:
        - sinfo #available nodes in various queues, queues are listed in "Hardware" section
        - squeue # queued jobs
        - scontrol show job=16 # show details for job 16
        - scancel 16 # cancel job 16

Test Cases Performed:

The experiments evaluated the execution time of SUMMA-A and SUMMA-C under varying conditions, including different matrix sizes, aspect ratios, and numbers of processing nodes.

- **Square Matrices:** The performance of SUMMA was tested on square matrices of increasing sizes to evaluate **scalability and efficiency**

Category	Matrix Dimensions

Small	4,096 x 4,096
Small	8,192 x 8,192
Medium	16,384 x 16,384
Medium	32,768 x 32,768
Large	65, 536 x 65,536

- **Rectangular Matrices:** For real-world applications, matrices often have highly asymmetric shapes (e.g., feature matrices in machine learning). These tests evaluated the impact of M > K.

Category	Matrix Dimensions
Small	4,096 x 128
Small	8,192 x 128
Medium	16,384 x 128
Medium	32,768 x 128
Large	65, 536 x 128

Verification of Correctness:

We have verified the correctness of the code with the following matrix dimensions:

Square Matrix:

- **N = 4096**
  Matrix Dimensions:
  A: 4096 x 4096
  B: 4096 x 4096
  C: 4096 x 4096
  Grid size: 4 x 4
  Block size: 16

Algorithm: Stationary a
Verbose: false

Verification Results:
Total Elements: 16777216
Errors: 0 (0.00%)
Max Error: 0.000000e+00 at position [-1,-1] (index -1)
Average Error: 0.000000e+00

Matrix Dimensions:
A: 4096 x 4096
B: 4096 x 4096
C: 4096 x 4096
Grid size: 4 x 4
Block size: 16
Algorithm: Stationary c
Verbose: false

Verification Results:
Total Elements: 16777216
Errors: 0 (0.00%)
Max Error: 0.000000e+00 at position [-1,-1] (index -1)
Average Error: 0.000000e+00

- **N = 8192**
  Matrix Dimensions:
  A: 8192 x 8192
  B: 8192 x 8192
  C: 8192 x 8192
  Grid size: 4 x 4
  Block size: 16
  Algorithm: Stationary c
  Verbose: false

  Verification Results:
  Total Elements: 67108864
  Errors: 0 (0.00%)
  Max Error: 0.000000e+00 at position [-1,-1] (index -1)

Average Error: 0.000000e+00

Matrix Dimensions:
A: 8192 x 8192
B: 8192 x 8192
C: 8192 x 8192
Grid size: 4 x 4
Block size: 16
Algorithm: Stationary c
Verbose: false

Verification Results:
Total Elements: 67108864
Errors: 0 (0.00%)
Max Error: 0.000000e+00 at position [-1,-1] (index -1)
Average Error: 0.000000e+00

Rectangular Matrix:

- **N = 4096**
  Matrix Dimensions:
  A: 4096 x 128
  B: 128 x 4096
  C: 4096 x 4096
  Grid size: 4 x 4
  Block size: 16
  Algorithm: Stationary a
  Verbose: false

  Verification Results:
  Total Elements: 16777216
  Errors: 0 (0.00%)
  Max Error: 0.000000e+00 at position [-1,-1] (index -1)
  Average Error: 0.000000e+00

  Matrix Dimensions:
  A: 4096 x 128
  B: 128 x 4096
  C: 4096 x 4096

Grid size: 4 x 4
Block size: 16
Algorithm: Stationary c
Verbose: false

Verification Results:
Total Elements: 16777216
Errors: 0 (0.00%)
Max Error: 0.000000e+00 at position [-1,-1] (index -1)
Average Error: 0.000000e+00

- **N = 8192**
  Matrix Dimensions:
  A: 8192 x 128
  B: 128 x 8192
  C: 8192 x 8192
  Grid size: 4 x 4
  Block size: 16
  Algorithm: Stationary a
  Verbose: false

  Verification Results:
  Total Elements: 67108864
  Errors: 0 (0.00%)
  Max Error: 0.000000e+00 at position [-1,-1] (index -1)
  Average Error: 0.000000e+00

  Matrix Dimensions:
  A: 8192 x 128
  B: 128 x 8192
  C: 8192 x 8192
  Grid size: 4 x 4
  Block size: 16
  Algorithm: Stationary c
  Verbose: false

  Verification Results:
  Total Elements: 67108864
  Errors: 0 (0.00%)

Max Error: 0.000000e+00 at position [-1,-1] (index -1)
Average Error: 0.000000e+00

Performance Analysis and Observations:

The execution time was recorded using gettimeofday() for the following components:

- Matrix block distribution (MPI_Scatter / MPI_Send)
- Computation time (matrix multiplication).
- Communication overhead (MPI_Bcast, MPI_Gatherv).
- Result gathering (MPI_Reduce / MPI_Gather).

Execution Time Analysis:

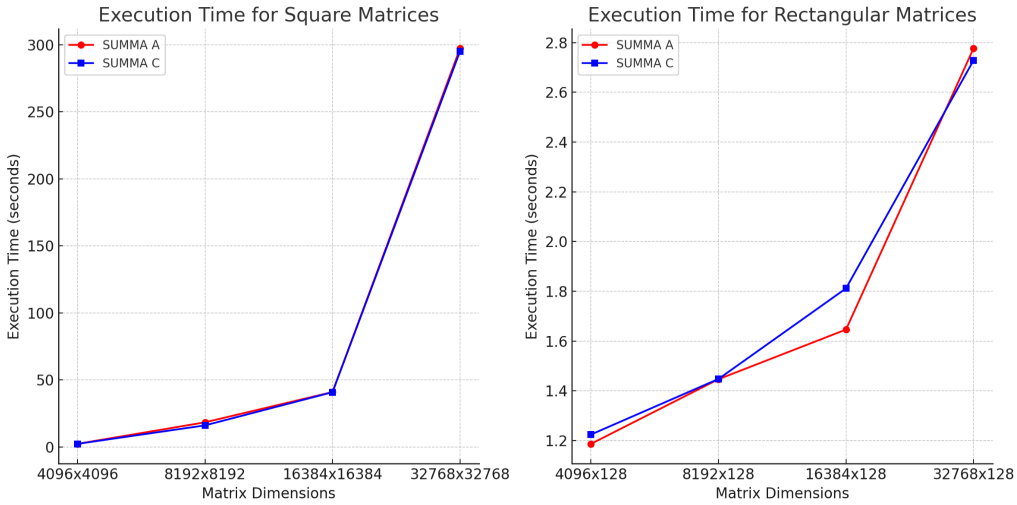
The recorded execution times for SUMMA-A and SUMMA-C across different matrix sizes are shown below.

Square Matrices Execution Time Analysis:

Matrix Size	**Processes**	**Block Size**	**SUMMA A (sec)**	**SUMMA C (sec)**
4096 x 4096	16	16	2.298	2.280
8192 x 8192	16	16	18.349	16.060
16384 x 16384	64	32	40.929	40.822
32768 x 32768	64	32	297.282	295.128

Rectangular Matrices Execution Time Analysis:

Matrix Size	Processes	Block Size	SUMMA A (sec)	SUMMA C (sec)
4096 x 128	16	16	1.186	1.223
8192 x 128	16	16	1.446	1.447
16384 x 128	64	32	1.645	1.811
32768 x 128	64	32	2.776	2.728



The execution time analysis compares the performance of SUMMA A and SUMMA C for square matrices and tall-skinny matrices. The results are visualized in the provided graphs, showing execution times across different matrix sizes.

For square matrices, both SUMMA A and SUMMA C exhibit similar performance trends, but SUMMA C is slightly more efficient in most cases:

- For smaller matrices (4096 × 4096), both algorithms perform almost identically (~2.3 seconds).
- For intermediate matrices (8192 × 8192), SUMMA C executes in 16.06 seconds, compared to 18.34 seconds for SUMMA A.
- For larger matrices (16384 × 16384 and 32768 × 32768), both algorithms take roughly 40.9 seconds and 297 seconds, respectively, with SUMMA C maintaining a slight edge.

SUMMA C broadcasts A along rows and B along columns, leading to better load balancing and fewer redundant communications. The communication pattern in SUMMA C is more structured, reducing latency and improving performance, especially as the number of processors increases.

For rectangular matrices, where matrix A has many rows but few columns:

- For small matrices (4096 × 128, 8192 × 128), SUMMA A performs slightly better, finishing in 1.18 seconds vs. 1.22 seconds for SUMMA C.
- For intermediate matrices (16384 × 128, 32768 × 128), SUMMA C becomes more efficient, outperforming SUMMA A by 0.1 to 0.05 seconds.

In SUMMA C, B is broadcast column-wise, making it more efficient when A is wide and shallow (as in tall-skinny cases). Load balancing is more optimized, ensuring an even

distribution of computations across all nodes. Memory locality is better managed due to the structured row-wise and column-wise broadcasts.

Memory Access Patterns:

In SUMMA A each process receives a subset of A and a broadcasted copy of B for each iteration. The memory access pattern focuses more on A being fixed and B being broadcast.

In SUMMA C the broadcasting of A along rows and B along column, improves cache reuse since multiple processes share the same submatrices across different iterations. SUMMA C reduces redundant memory accesses by efficiently reusing broadcasted data, leading to less memory congestion in larger matrices.

Factors Affecting Load Balance:

Load balancing in parallel matrix multiplication is important for performance. The key factors influencing load balance in SUMMA A and SUMMA C include:

- Grid Size.
- Communication Overhead.
- Matrix Dimensions.
- Cache efficiency.

Conclusion:

SUMMA C generally outperforms SUMMA A, particularly for large square matrices and tall-skinny matrices. For square matrices, SUMMA C performs marginally better due to its optimized row-wise and column-wise broadcasts reducing communication, redundancy.

For tall-skinny matrices, SUMMA C exhibits better load balancing and efficient memory access patterns, which results in faster execution times. The difference between the two methods is not drastic, but SUMMA C consistently exhibits slightly better efficiency due to optimized communication strategies.

Given that both approaches exhibit similar performance, selecting SUMMA C was preferable for better communication efficiency. By structuring broadcast operations in SUMMA C, memory access patterns were improved, and unnecessary data transfers were minimized. Since both SUMMA A and SUMMA C scale well with increasing matrix sizes, the impact of efficient

communication becomes more pronounced at larger scales. Based on this execution time analysis, **SUMMA C** is the preferred approach for large-scale matrix multiplications, particularly when communication efficiency plays a crucial role.